

Industrial Internship Report on "Prediction of Agriculture Crop Production in India"

**Prepared by
Guradesh Dhillon**

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was Prediction of Agriculture Crop Production in India, where I built a machine learning regression pipeline to forecast crop production (in tons/quintals) across Indian states using historical data on crop type, variety, cultivated area, season, and cultivation cost, sourced from data.gov.in.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1	Preface.....	3
2	Introduction.....	4
2.1	About UniConverge Technologies Pvt Ltd.....	4
2.2	About upskill Campus.....	8
2.3	Objective.....	9
2.4	Reference.....	9
2.5	Glossary.....	10
3	Problem Statement.....	11
4	Existing and Proposed solution.....	12
5	Proposed Design/ Model.....	13
5.1	High Level Diagram (if applicable).....	13
5.2	Low Level Diagram (if applicable).....	13
5.3	Interfaces (if applicable).....	13
6	Performance Test.....	14
6.1	Test Plan/ Test Cases.....	14
6.2	Test Procedure.....	14
6.3	Performance Outcome.....	14
7	My learnings.....	15
8	Future work scope.....	16

1 Preface

Over the 6-week internship, I worked through the fundamentals of data science and machine learning workflows: exploratory data analysis, data cleaning, model building, and evaluation, culminating in a complete crop production prediction pipeline built and tested on Google Colab and Antigravity IDE.

As a third-year Computer Engineering student specializing in AI & Data Science, hands-on internships like this one are essential for bridging the gap between classroom theory and real, industry-relevant problem solving, and for building a credible project portfolio.

The assigned problem was to predict agricultural crop production across Indian states using historical cultivation data, in order to support better planning around crop selection, resource allocation, and cost estimation for farmers and policymakers.

upSkill Campus and UniConverge Technologies gave me the opportunity to work on a real, industry-relevant machine learning problem instead of a purely academic exercise, along with structured weekly guidance and mentorship throughout the program.

The 6-week program was structured week by week: onboarding and Python/ML fundamentals, dataset selection, exploratory data analysis, model building and iteration, performance evaluation, and finally documentation and submission of the code and report on GitHub.



This internship strengthened my understanding of the full ML workflow, from raw, messy tabular data to a deployable regression model, and taught me the value of comparing multiple models rather than trusting a single one. Overall, it was a highly rewarding, practical experience.

I would like to thank the entire upSkill Campus and UniConverge Technologies team, and The IoT Academy, for structuring this internship and providing the guidance and resources needed to complete this project successfully.

To juniors attempting this internship: start with thorough EDA before jumping to modeling, always compare at least two models rather than settling for the first one, and keep your GitHub updated as you go rather than leaving documentation for the last day.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT)**, **Cyber Security**, **Cloud computing (AWS, Azure)**, **Machine Learning**, **Communication Technologies (4G/5G/LoRaWAN)**, **Java Full Stack**, **Python**, **Front end** etc.



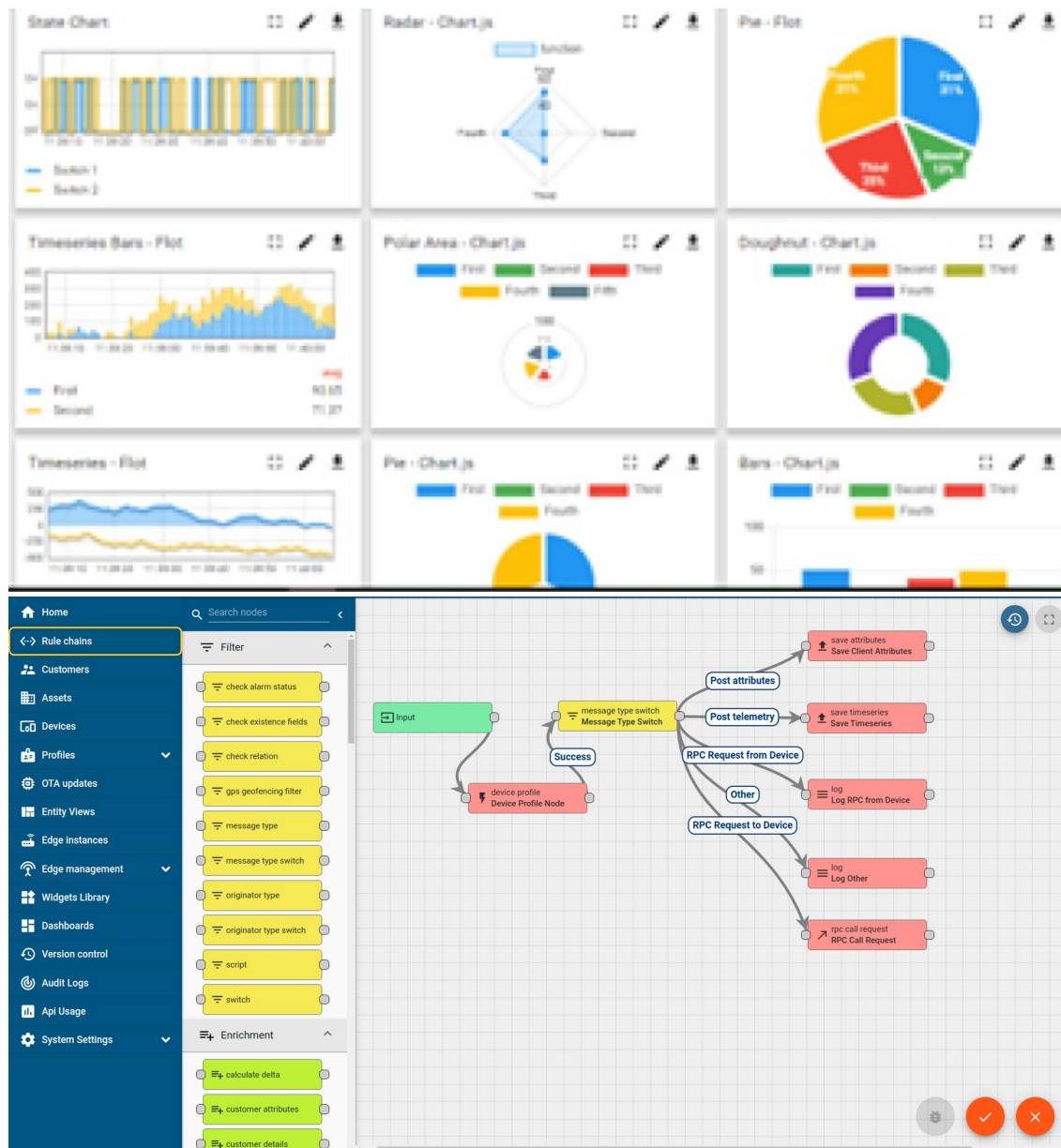
i. UCT IoT Platform (**Insight**)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY **WATCH**

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



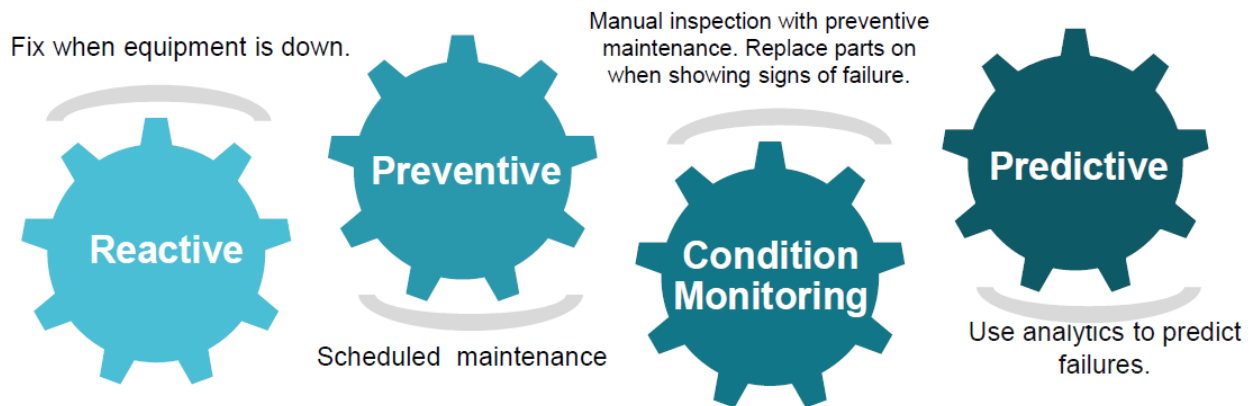


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.

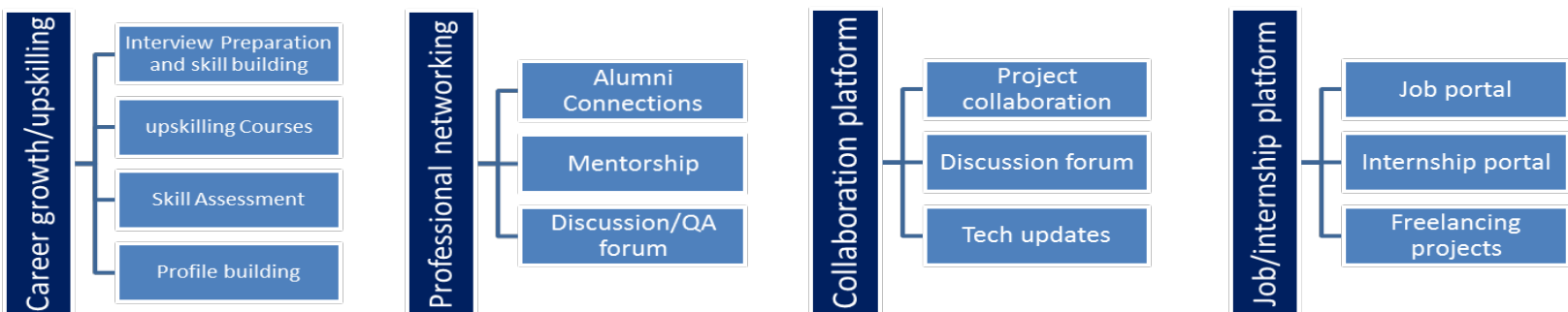


2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.





2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

- [1] Crop Production dataset, data.gov.in (Open Government Data Platform India)
- [2] scikit-learn: Machine Learning in Python, <https://scikit-learn.org/stable/>
- [3] upSkill Campus, <https://www.upskillcampus.com/>

2.6 Glossary

Terms	Acronym
ML	Machine Learning
EDA	Exploratory Data Analysis
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
R2	Coefficient of Determination (R-squared)

3 Problem Statement

In the assigned problem statement

In the assigned problem statement, I worked on Project 4: Prediction of Agriculture Crop Production in India. Crop production across India varies significantly by state, district, crop type, season, and cultivated area, which makes planning difficult for farmers, cooperatives, and policymakers. Existing estimation methods rely heavily on historical averages and manual projection, with no simple, ready-to-use model that estimates expected production from basic cultivation parameters. The task was to build a machine learning regression model on real historical cultivation data sourced from data.gov.in (246,091 records covering 33 states, 646 districts, 124 crops, and the years 1997-2015) that predicts crop production, given inputs such as state, crop, season, cultivated area, and year, and to evaluate how reliably this model performs against a simple baseline method.

4 Existing and Proposed solution

Existing approaches to estimating crop production in India largely rely on historical averages, government survey-based projections, and manual estimation by agricultural officers based on past seasons. These are useful but slow to update, do not capture interactions between variables (for example, how cost of cultivation and area jointly affect production for a specific crop-state combination), and do not scale well across the large number of crop-state-season combinations that exist in practice.

My proposed solution is a supervised machine learning regression pipeline that: (1) cleans and pre-processes the raw agricultural dataset (246,091 records from data.gov.in), handling missing production values, duplicates, and invalid area entries; (2) applies a log1p transform to Area and Production, since raw production values are extremely right-skewed due to mixed units across crops (tonnes, nuts, bales, bunches); (3) encodes categorical variables (State, Crop, Season) using one-hot encoding; (4) trains and compares two models, a Linear Regression baseline and a Random Forest Regressor, to predict production; (5) selects the best-performing model based on real-unit MAE and RMSE (not log-scale R-squared alone, since the two metrics disagreed for this dataset); and (6) reports feature importance to identify which cultivation factors most strongly drive production.

Compared to manual estimation, this approach can quickly generate production estimates for any crop-state-season combination present in the training distribution, is retrainable as new data becomes available, and quantifies which cultivation factors matter most, giving farmers and policymakers actionable insight rather than a single opaque number.

4.1 Code submission (Github link)

https://github.com/guradeshdhillon/upskillCampus/blob/main/crop_production_forecast.py

4.2 Report submission (Github link)

https://github.com/guradeshdhillon/upskillCampus/blob/main/CropProduction_Guradesh_USC_UCT.pdf

4.3

5 Proposed Design/ Model

Given more details about design flow of your solution. This is applicable for all domains. DS/ML Students can cover it after they have their algorithm implementation. There is always a start, intermediate stages and then final outcome.

5.1 High Level Diagram (if applicable)

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

The pipeline flows as follows: Raw CSV Data (Crop, Variety, State, Year, Season, Area, Cost, Production) -> Data Cleaning and Validation (duplicates removed, missing values imputed, invalid negative values dropped) -> Feature Engineering (one-hot encoding of categorical columns via a ColumnTransformer) -> Train/Test Split (80/20) -> Model Training (Linear Regression baseline and Random Forest Regressor, trained in parallel) -> Model Evaluation (MAE, RMSE, R-squared on held-out test data) -> Best Model Selection -> Saved Model (crop_production_model.joblib) -> Inference on new data.

5.2 Low Level Diagram (if applicable)

Within the pipeline: the Data Layer uses pandas for loading, cleaning, and validating the CSV; the Feature Layer uses scikit-learn's ColumnTransformer and OneHotEncoder to convert categorical fields (Crop, Variety, State, Season) into numeric features while passing numeric fields (Year, Area, Cost) through unchanged; the Model Layer wraps preprocessing and the estimator together in a single sklearn Pipeline object, so the same object handles both training and inference consistently; and the Persistence Layer serializes the fitted pipeline using joblib so it can be reloaded and reused without retraining.

5.3 Interfaces (if applicable)

Input interface: a CSV file with columns Crop, Variety, State, Year, Season, Area_Hectares, and Cost.

Output interface: a single predicted Production value (numeric, in quintals/tons) plus a ranked feature-importance list showing which inputs most influenced the prediction. The trained pipeline exposes a single predict() call, so it could be wrapped behind a simple REST API (for example FastAPI) for real-time inference in future iterations of this project.

Update with Block Diagrams, Data flow, protocols, FLOW Charts, State Machines, Memory Buffer Management.

6 Performance Test

This is very important part and defines why this work is meant of Real industries, instead of being just academic project.

Here we need to first find the constraints.

How those constraints were taken care in your design?

What were test results around those constraints?

Constraints can be e.g. memory, MIPS (speed, operations per second), accuracy, durability, power consumption etc.

In case you could not test them, but still you should mention how identified constraints can impact your design, and what are recommendations to handle them.

6.1 Test Plan/ Test Cases

The test plan used an 80/20 train-test split (random_state=42, for reproducibility) on the full 246,091-row data.gov.in dataset (242,361 rows after cleaning). Two models were evaluated on the same held-out test set: a Linear Regression baseline and a Random Forest Regressor. Additional test cases confirmed the pipeline handles missing production values (3,730 rows), duplicate rows, zero/negative Area entries, and trailing whitespace in the Season column without crashing, and that categorical values not seen during training are safely ignored at inference time (via handle_unknown="ignore" in the encoder) rather than raising an error.

6.2 Test Procedure

After training each model on the training split (on a log1p-transformed target, to handle the extreme right-skew caused by mixed units across crops), predictions were converted back to real production units (via expm1) and compared against actual production values using three metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (coefficient of determination, measured on the log scale). Feature importance was additionally extracted from the Random Forest model to identify which cultivation factors contributed most to its predictions.

6.3 Performance Outcome

On the full data.gov.in dataset (242,361 cleaned records), the two models gave an interesting split result. Linear Regression achieved a marginally higher log-scale R-squared (0.9335 versus 0.9212 for Random Forest), but Random Forest achieved a dramatically lower real-unit error: MAE of 135,686.00 versus

671,783.62 for Linear Regression, and RMSE of 6,481,035.31 versus 19,595,673.35. Because R-squared on the log scale rewards getting the overall pattern right while MAE in real units reflects actual prediction accuracy for a given crop-state-season record, Random Forest was selected as the final model based on MAE. Area under cultivation (log-transformed) was by far the strongest predictor of production (feature importance 0.840), followed by specific high-yield-variance crops such as Sugarcane (0.044) and Coconut (0.022). This result was itself an important learning: relying on a single metric (R-squared) without checking others (MAE, RMSE) could have led to selecting the less accurate model.

7 My learnings

This internship gave me hands-on experience with the complete ML lifecycle, not just model training in isolation. Key takeaways: (1) data cleaning and validation take up a large share of real project time, and skipping them quietly corrupts model quality; (2) comparing multiple models against clear metrics, rather than trusting one, is essential to justify design choices; (3) feature importance analysis turns a model from a black-box number generator into something that produces actionable, explainable insight; (4) structuring code as a reusable, serializable pipeline (via joblib) makes it far easier to deploy or hand off later; and (5) different evaluation metrics can disagree on which model is "best", so choosing a metric that reflects the real-world use case (real-unit MAE, in this case) matters more than optimizing a single abstract number like R-squared. This experience strengthens my applied ML skill set alongside my earlier project work, and gives me more confidence tackling unfamiliar, messy real-world datasets end-to-end.

8 Future work scope

Given more time, I would extend this project by: (1) incorporating additional real-world features such as rainfall, soil type, and irrigation availability, which are known to significantly affect crop yield; (2) trying gradient boosting models such as XGBoost or LightGBM and comparing them against Random Forest; (3) building a small web interface, for example using FastAPI or Streamlit, so a farmer or policymaker could input crop, state, season, and area and get an instant production estimate; and (4) extending the model to forecast multiple years ahead using time-series techniques, since the current model treats Year as a static feature rather than modeling temporal trends explicitly.